Comment implémenter

🎯 Objectif de la phase IMPLÉMENTER

La phase **IMPLÉMENTER** est le cœur du **FORGE** où les stories déjà prêtes, découpées et validées en **PLANIFIER** sont codées, intégrées, testées et documentées. La spécification devient du code fonctionnel, conforme et prouvé.

> Implémenter, ce n'est pas juste coder. C'est traduire fidèlement les spécifications en code, en tests et en preuves de conformité — sans interpréter, sans deviner, sans improviser.

Principe fondamental : les **spécifications atomiques issues de DÉFINIR sont la seule source de vérité**. Le développeur code ce qui est spécifié. Le QA teste ce qui est spécifié. L'intégrateur valide ce qui est spécifié. Tout le monde travaille à partir du même artefact.

> **Note terminologique.** Cette page décrit la phase **IMPLÉMENTER** dans son contexte opérationnel. Pour les définitions de référence des termes utilisés ici — spécification IF… THEN…, spécification atomique, DoR, DoD, validation, vérification, test, niveaux d'intégration, interfaces, angles d'observation, intégration, contrat d'intégration, CI/CD, artefact, dette technique, qualimétrie, rôles et RACI — voir le Lexique / Terminologies et la page **SPÉCIFICATION**.
>
> Les descriptions locales précisent l'usage de ces notions pendant IMPLÉMENTER. Les contributions indiquées dans cette page sont propres à cette phase et ne remplacent pas la matrice RACI. Les anciens termes comme critère d'acceptation, cas de test ou scénario convergent ici vers la spécification IF… THEN… et ne doivent pas recréer une couche séparée.

🔀 Le passage de DÉFINIR / PLANIFIER à IMPLÉMENTER

Avant de commencer, les prérequis suivants doivent être satisfaits :

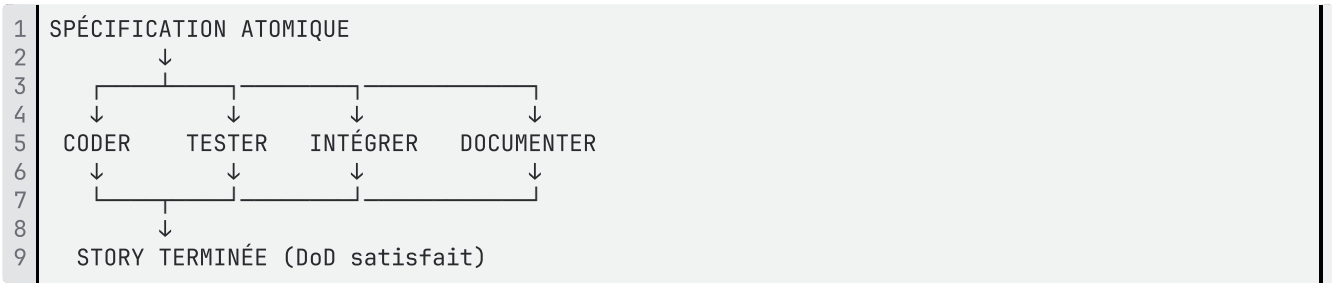
Prérequis	Vérification
FEATURES découpées en stories	Chaque story est claire, cohérente et traçable à sa spécification atomique.
Couverture des spécifications	On sait exactement quelles spécifications atomiques sont couvertes par chaque story.

Stories estimées finement	Effort, complexité et capacité validés par l'équipe.
Priorités validées par le PO	Selon la valeur, le risque et les dépendances.
DoR satisfait	Clarté, testabilité, dépendances gérées, estimation validée.
Pré-requis techniques validés	Environnements, outils, accès, données et configurations disponibles.
Dépendances planifiées	Aucune dépendance bloquante non résolue.

> Si le DoR n'est pas satisfait, la story ne rentre pas en implémentation. C'est une règle, pas une suggestion.

🛠️ Ce que « implémenter » veut dire concrètement

Implémenter couvre quatre activités indissociables qui se font en parallèle, pas en séquence :



1. 💻 CODER — Traduire la spécification en code

- Le code est une **traduction directe** de la spécification atomique, pas une interprétation.
- Le développeur implémente exactement ce qui est spécifié : ni plus, ni moins.
- Si la spécification est ambiguë ou incomplète, on ne devine pas : on retourne à DÉFINIR.
- Le code respecte les standards de développement, de sécurité et de conformité du Contrat qualité.
- Les revues de code sont effectuées et approuvées par les pairs.

2. 🧪 TESTER — Prouver la conformité à tous les niveaux

- Les validations et tests sont dérivés directement des **spécifications IF... THEN...**, sans couche séparée de critères d'acceptation.
- Les tests suivent l'approche **RED4GREEN** : écrire ou ajuster le test qui échoue d'abord, puis implémenter jusqu'à obtenir un résultat vert.

- Les tests couvrent les niveaux pertinents : **Small**, **Medium** et **Large / E2E**.
- Les approches whitebox, blackbox et graybox peuvent s'appliquer à tous les niveaux.
- La couverture unitaire minimale est de **60 %** lorsque mesurable.
- Les **spécifications critiques** doivent avoir au moins une validation associée et un résultat documenté.
- L'automatisation est la règle ; le manuel est l'exception justifiée. Les tests automatisables doivent être intégrés au pipeline lorsque pertinent.

Niveau	Ce qu'on teste	Ce qu'on prouve
Small	Le plus petit comportement isolable.	La logique interne fonctionne correctement.
Medium	L'intégration entre composants.	Les contrats, échanges et dépendances fonctionnent ensemble.
Large / E2E	Le comportement complet du point de vue utilisateur ou système.	Le flux de bout en bout produit le résultat attendu.

3. 🔗 INTÉGRER — Assembler et valider les contrats

- Le code est intégré dans la branche principale ou la branche de release appropriée.
- Les vérifications CI/CD passent sans erreur.
- Les contrats d'intégration sont respectés et validés avec les systèmes partenaires.
- Les dépendances inter-équipes sont intégrées et testées.
- Les artefacts de build sont générés, versionnés et stockés dans les dépôts officiels.

4. 📝 DOCUMENTER — Maintenir la traçabilité vivante

- La documentation technique est mise à jour : code, API, configuration, changelog.
- La documentation est liée aux spécifications, pas gérée comme un document séparé en silo.
- La traçabilité **spécification → code → tests → résultats** est maintenue.
- Les risques, anomalies critiques et dettes découverts pendant l'implémentation sont documentés et remontés au backlog **ÉLICITER** pour préserver la boucle de rétroaction.

🔄 La boucle spécification → code → test

Le code et les tests sont deux faces de la même spécification :

```
SPÉCIFICATION ATOMIQUE
     ↙             ↘
  CODE             TESTS
     ↘             ↙
    CONCORDANCE 100%
```

- Le code implémente le comportement décrit dans la spécification.
- Les tests vérifient que le code respecte la spécification.
- Si le test échoue, soit le code est incorrect, soit la spécification doit être clarifiée.
- Il n'y a jamais de code sans test, ni de test sans spécification.

> Si tu ne sais pas quoi tester, c'est que la spécification n'est pas assez claire. Si tu ne sais pas quoi coder, c'est pareil.

🤖 IA et automatisation pendant l'implémentation

Les spécifications atomiques solides permettent d'utiliser l'IA comme levier pendant l'implémentation :

Levier	Application concrète
Génération de code assistée	L'IA génère du code conforme à la spécification ; le développeur valide et ajuste.
Génération de tests automatisée	L'IA dérive des validations automatisées à partir des **spécifications IF... THEN...** et des exemples.
Auto-validation	Vérification automatique que le code respecte la spécification.
Détection de régressions	L'IA identifie les impacts potentiels d'un changement sur les tests existants.
Revue de code assistée	L'IA détecte les incohérences entre le code et la spécification.

> Test de qualité : est-ce que l'IA peut générer le code ET les tests à partir de la spécification seule ? Si non, on retourne à DÉFINIR.

👥 Qui fait quoi pendant IMPLÉMENTER ?

Rôle	Contribution
Développeur	Code, revue de code, tests unitaires, intégration et documentation technique.
QA / Ingénieur en validation	Tests d'intégration, tests fonctionnels, automatisation et validation des **spécifications IF... THEN...**.
PO	Valide que l'implémentation correspond à la spécification et à la valeur attendue.
Architecte	Valide les contrats d'intégration, les impacts systémiques et la conformité architecturale.
Support / CARE	Prépare le transfert, identifie les risques opérationnels et valide la documentation utile au support.
Chargé de livraison (CdL)	Surveille le flux, la conformité au SDLC et prépare les décisions pour le VSE.

🤝 Pair testing pendant IMPLÉMENTER

Le pair testing sert à traiter les zones grises d'intégration avant la diffusion, lorsque le comportement réel dépend de plusieurs composants, environnements ou responsabilités.

- Planifier des sessions ciblées **DEV × QA**, **DEV × IVV**, **QA × IVV** ou **dev/ops** selon les risques de la story.
- Utiliser le pair testing pour valider les contrats d'intégration, les cas limites, les transitions d'état, les prérequis d'environnement et les comportements difficiles à automatiser immédiatement.
- Documenter les constats, décisions, anomalies et apprentissages dans Jira ou dans la preuve de validation associée.
- Remonter à **DÉFINIR** les ambiguïtés de spécification et à **ÉLICITER** les nouveaux besoins, risques ou signaux qualimétrie.

✅ Definition of Done (DoD) — Quand une story est terminée

Critère DoD	Preuve attendue

Le code implémente la spécification atomique	Revue de code approuvée.
Tests unitaires (auto) RED4GREEN	Rapport de couverture ≥ 60 % lorsque mesurable et résultats automatisés disponibles.
Tests d'intégration (auto) RED4GREEN	Résultats documentés et intégrés au pipeline lorsque pertinent.
Spécifications fonctionnelles (auto) RED4GREEN	Chaque spécification IF… THEN… critique a un résultat documenté.
Le code passe les vérifications CI/CD	Build vert, aucune erreur bloquante.
Les contrats d'intégration sont respectés	Validation inter-équipes documentée.
Les revues de code sont effectuées et approuvées	Approbations enregistrées dans l'outil de revue.
La documentation technique est à jour	Documentation liée aux spécifications.
Les anomalies critiques sont corrigées ou remontées au backlog (ÉLICITER)	Correction validée ou plan de résolution documenté.
La dette technique découverte est documentée et remontée au backlog (ÉLICITER)	Dette qualifiée, impact évalué et item de backlog créé.
Les artefacts de build sont générés et versionnés	Stockés dans les dépôts officiels.
Le DoD est satisfait pour chaque story, dépendance et epic concerné	Vérification explicite au niveau story, dépendances et epic.

> Si un seul critère n'est pas satisfait, la story n'est pas terminée. Pas de « presque fini ».

🔁 Boucles de rétroaction pendant IMPLÉMENTER

L'implémentation n'est pas un tunnel : elle génère des signaux qui doivent remonter.

```
DÉFINIR ← signaux, incohérences, cas manquants ← IMPLÉMENTER → métriques, risques → ÉLICITER
                                                        ↓
                                                    DIFFUSER
```

Signal	Action

Spécification ambiguë ou incomplète	Retour à DÉFINIR pour clarification ; on ne devine pas.
Cas limite non spécifié découvert	Documenter et remonter à DÉFINIR pour enrichir la spécification.
Dette technique bloquante découverte	Documenter, évaluer l'impact et remonter au backlog produit.
Risque opérationnel identifié	Communiquer au CdL et au PO pour arbitrage.
Métriques qualimétrie révélant une tendance	Remonter comme signal à ÉLICITER pour la boucle d'amélioration continue.

> Si en implémentant on découvre que la spécification ne couvre pas un cas, ce n'est pas un échec : c'est le système qui fonctionne. On remonte, on clarifie, et on revient.

📊 Métriques et signaux

La phase IMPLÉMENTER doit produire des preuves mesurables de conformité, de stabilité et de maîtrise du risque. Ces signaux alimentent la décision de passage vers DIFFUSER et la boucle d'amélioration continue.

Métrique / Signal	Ce qu'elle révèle	Source
Couverture des tests et preuves de conformité	Niveau de preuve associé au code livré et aux **spécifications IF... THEN...**.	CI/CD, rapports de tests, SonarQube
Taux de réussite des pipelines CI/CD	Stabilité de l'intégration et maturité de la chaîne de livraison.	CI/CD
Nombre d'anomalies critiques ou bloquantes ouvertes	Risque résiduel avant passage à DIFFUSER.	Jira, outil de suivi des défauts
Dette technique détectée pendant l'implémentation	Coût futur potentiel et sujets à remonter au backlog produit.	Revues de code, SonarQube, backlog

Écarts spécification → code → tests	Manque de traçabilité ou divergence entre ce qui était défini et ce qui est livré.	Revues, matrice de traçabilité, tests
Temps moyen de correction des défauts	Capacité de l'équipe à réagir aux problèmes découverts pendant l'implémentation.	Jira, CI/CD

⚠️ Pièges courants à éviter

Piège	Pourquoi c'est un problème	Comment l'éviter
Coder sans spécification approuvée	On implémente quelque chose qui n'a pas été validé ; retravail garanti.	Exiger le SPECIFY approuvé et le DoR satisfait avant de commencer.
Interpréter la spécification	Le dev « comprend » différemment du QA ; divergence assurée.	Coder exactement ce qui est spécifié. En cas de doute, retourner à DÉFINIR.
Tester après avoir codé en séquence	Les tests deviennent une formalité, pas une validation.	Coder et tester en parallèle ; les tests sont dérivés de la spécification.
Ignorer les tests d'intégration	Les composants fonctionnent seuls mais pas ensemble.	Valider les contrats d'intégration systématiquement.
Ne pas documenter la dette découverte	La dette s'accumule silencieusement.	Documenter et remonter immédiatement au backlog.
Dire « presque fini »	Une story à 95 % n'est pas terminée ; elle est en cours.	Appliquer le DoD strictement : terminé ou pas terminé.
Coder des fonctionnalités non spécifiées	Du code non demandé = du code non testé = de la dette.	Implémenter uniquement ce qui est dans la spécification atomique.
Ne pas remonter les signaux	Les problèmes découverts en implémentation sont perdus.	Utiliser la boucle de rétroaction vers DÉFINIR et ÉLICITER.

✅ Critères de sortie de IMPLÉMENTER

La phase IMPLÉMENTER est terminée quand :

- Toutes les stories planifiées sont implémentées conformément à leurs **spécifications IF... THEN...**.
- Le code source respecte les standards de développement, de sécurité et de conformité du Contrat qualité.
- Tests unitaires (auto) RED4GREEN réalisés et verts, avec seuil de couverture minimal atteint lorsque mesurable.
- Tests d'intégration (auto) RED4GREEN réalisés et verts lorsque pertinent.
- Spécifications fonctionnelles (auto) RED4GREEN validées : les **spécifications IF... THEN...** sont satisfaites pour chaque story.
- Les anomalies critiques ou bloquantes sont corrigées ou remontées au backlog **ÉLICITER** avec un plan de résolution.
- Les dépendances techniques et inter-équipes sont intégrées et validées.
- Les contrats d'intégration sont respectés et les interfaces validées.
- Le pair testing requis est réalisé sur les zones grises d'intégration et les constats sont documentés.
- Le code est intégré dans la branche principale et passe les vérifications CI/CD sans erreur.
- Les artefacts de build sont générés, versionnés et stockés dans les dépôts officiels.
- La documentation technique est mise à jour et liée aux spécifications.
- Les revues de code sont effectuées et approuvées.
- Les risques et dettes identifiés sont documentés et remontés au backlog **ÉLICITER**.
- Le DoD est satisfait pour chaque story, dépendance et epic concerné.
- Les environnements de test et de validation sont stables et prêts pour la phase DIFFUSER.

🔄 Lien avec les autres phases

```
DÉFINIR → spécifications atomiques → PLANIFIER → stories prêtes (DoR) → IMPLÉMENTER → code validé (DoD) → DIFFUSER
   ↑                                                                                    |
   └──── signaux, cas manquants, incohérences, dette technique ──────────────────────┘
```

IMPLÉMENTER est la traduction fidèle des spécifications en code prouvé. Quand cette phase est bien faite, DIFFUSER reçoit un build solide, testé et documenté. Quand elle est bâclée, la dette se paie en bugs, en retravail et en incidents post-livraison.

> On ne peut pas bien diffuser ce qu'on n'a pas bien implémenté. Et on ne peut pas bien implémenter ce qu'on n'a pas bien défini.

Références

- Cérémonie d'implémentation
- Comment planifier — phase précédente
- Comment diffuser — phase suivante
- (SDLC) Software Development Life Cycle (officiel)
- Contrat qualité officiel OPAL-RT
- RACI — Rôles et responsabilités
- Qualimétrie officielle OPAL-RT
- SPÉCIFICATION
- Les tests (niveau × interface × angle × type)
- Exemples concrets de combinaisons
- Lexique / Terminologies officiel OPAL-RT
- Recommandations — enjeux, constats ou lacunes à résoudre / solutionner